Bab 10
Autentikasi dan Otorisasi API dengan JWT

Autentikasi dan otorisasi API dengan JWT (JSON Web Token) adalah pendekatan yang banyak digunakan dalam aplikasi modern untuk mengamankan komunikasi antara klien dan server, terutama pada arsitektur berbasis RESTful API.

Autentikasi dengan JWT dimulai ketika pengguna mengirimkan kredensial seperti email dan password ke endpoint login. Jika kredensial tersebut valid, server akan membuat dan mengirimkan token JWT ke pengguna. Token ini berisi informasi terenkripsi seperti ID pengguna dan masa berlaku token. Klien kemudian menyimpan token tersebut, biasanya di local storage atau session, dan menyertakannya pada setiap permintaan ke API menggunakan header Authorization dengan format Bearer.

JWT bersifat stateless, artinya server tidak menyimpan status sesi pengguna. Semua informasi yang diperlukan untuk memverifikasi permintaan sudah terdapat di dalam token itu sendiri. Inilah yang membuat JWT sangat efisien dan cocok untuk sistem terdistribusi seperti aplikasi mobile atau frontend berbasis JavaScript.

Otorisasi dilakukan dengan memverifikasi token yang dikirim. Saat klien mengakses endpoint yang memerlukan hak tertentu, server akan memeriksa token untuk memastikan apakah pengguna memiliki izin untuk melakukan tindakan tersebut. JWT juga dapat menyimpan data tambahan seperti peran pengguna (role), sehingga otorisasi berbasis peran dapat dilakukan tanpa perlu query tambahan ke database.

Dalam Laravel, JWT dapat diintegrasikan menggunakan paket pihak ketiga seperti tymon/jwt-auth. Paket ini menyediakan fitur lengkap untuk membuat, mengelola, dan memverifikasi token JWT, serta middleware untuk mengamankan route API.

Dengan autentikasi dan otorisasi menggunakan JWT, API Anda menjadi lebih aman, efisien, dan fleksibel. JWT sangat cocok digunakan ketika Anda mengembangkan API yang akan diakses dari berbagai platform dan tidak ingin mengelola sesi pengguna di sisi server.

1. Create Project:
   ```
   composer create-project laravel/laravel jwt-auth-app
   cd jwt-auth-app
   ```

2. Install Package JWT:
   ```
   composer require tymon/jwt-auth
   ```

3. Publish Konfigurasi JWT:
   ```
   php artisan vendor:publish --provider="Tymon\JWTAuth\Providers\LaravelServiceProvider"
   ```

4. Generate secret key untuk JWT:
   ```
   php artisan jwt:secret
   ```

5. Konfigurasi database .env

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=jwt_auth_app
DB_USERNAME=root
DB_PASSWORD=
```

6. Konfigurasi model App\Models\User:
    a. Buka file app/Models/User.php
    b. Ubah untuk implement JWT:

```php
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Tymon\JWTAuth\Contracts\JWTSubject;

class User extends Authenticatable implements JWTSubject
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'name',
        'email',
        'password',
        'role'

    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }
```

```
    public function getJWTIdentifier()
    {
        return $this->getKey();
    }

    public function getJWTCustomClaims()
    {
        return [];
    }
}
```

7. Buat Migrasi untuk Role

   php artisan make:migration add_role_to_users_table

   file: 2025_06_14_014331_add_role_to_users_table

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up()
    {
        Schema::table('users', function (Blueprint $table) {
            $table->enum('role', ['user', 'admin'])->default('user');
        });
    }

    public function down()
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

   Jalankan Migrasi

   php artisan migrate

8. Buat controller dengan nama AuthController

   php artisan make:controller AuthController

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use App\Models\User;
use Illuminate\Support\Facades\Validator;
use Tymon\JWTAuth\Facades\JWTAuth;

class AuthController extends Controller
{
    public function __construct()
```

```
    {
        $this->middleware('auth:api', ['except' => ['login', 'register']]);
    }

    public function login(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'email' => 'required|email',
            'password' => 'required|string|min:6',
        ]);

        if ($validator->fails()) {
            return response()->json($validator->errors(), 422);
        }

        if (!$token = JWTAuth::attempt($validator->validated())) {
            return response()->json(['error' => 'Unauthorized'], 401);
        }

        return $this->createNewToken($token);
    }

    public function register(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'name' => 'required|string|between:2,100',
            'email' => 'required|string|email|max:100|unique:users',
            'password' => 'required|string|confirmed|min:6',
            'role' => 'sometimes|in:user,admin'
        ]);

        if ($validator->fails()) {
            return response()->json($validator->errors()->toJson(), 400);
        }

        $user = User::create(array_merge(
            $validator->validated(),
            ['password' => bcrypt($request->password)]
        ));

        return response()->json([
            'message' => 'User successfully registered',
            'user' => $user
        ], 201);
    }

    public function logout()
    {
        auth()->logout();
        return response()->json(['message' => 'User successfully signed out']);
    }

    public function refresh()
    {
        try {
            $newToken = JWTAuth::parseToken()->refresh();
            return $this->createNewToken($newToken);
        } catch (\Exception $e) {
            return response()->json([
                'error' => 'Could not refresh token',
                'message' => $e->getMessage()
            ], 401);
        }
    }

    public function userProfile()
```

```
    {
        return response()->json(auth()->user());
    }

    protected function createNewToken($token)
    {
        return response()->json([
            'access_token' => $token,
            'token_type' => 'bearer',
            'expires_in' => JWTAuth::factory()->getTTL() * 60,
            'user' => auth()->user()
        ]);
    }

    // Admin only method
    public function adminDashboard()
    {
        if (auth()->user()->role !== 'admin') {
            return response()->json(['error' => 'Forbidden'], 403);
        }

        return response()->json([
            'message' => 'Welcome to Admin Dashboard',
            'stats' => [
                'users' => User::count(),
                'admins' => User::where('role', 'admin')->count()
            ]
        ]);
    }
}
```

9. Khusus versi terbaru:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use App\Models\User;
use Illuminate\Support\Facades\Validator;
use Tymon\JWTAuth\Facades\JWTAuth;

//HAPUS
use Illuminate\Routing\Middleware;


#[Middleware('auth:api', except: ['login', 'register'])]

class AuthController extends Controller
{
    //INI DIHAPUS:
    //public function __construct()
    //{
    //    $this->middleware('auth:api', ['except' => ['login', 'register']]);
    //}

    public function login(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'email' => 'required|email',
            'password' => 'required|string|min:6',
        ]);
```

```
        if ($validator->fails()) {
            return response()->json($validator->errors(), 422);
        }

        if (!$token = JWTAuth::attempt($validator->validated())) {
            return response()->json(['error' => 'Unauthorized'], 401);
        }

        return $this->createNewToken($token);
    }

    public function register(Request $request)
    {
        $validator = Validator::make($request->all(), [
            'name' => 'required|string|between:2,100',
            'email' => 'required|string|email|max:100|unique:users',
            'password' => 'required|string|confirmed|min:6',
            'role' => 'sometimes|in:user,admin'
        ]);

        if ($validator->fails()) {
            return response()->json($validator->errors()->toJson(), 400);
        }

        $user = User::create(array_merge(
            $validator->validated(),
            ['password' => bcrypt($request->password)]
        ));

        return response()->json([
            'message' => 'User successfully registered',
            'user' => $user
        ], 201);
    }

    public function logout()
    {
        Auth::logout();
        return response()->json(['message' => 'User successfully signed out']);
    }

    public function refresh()
    {
        try {
            $newToken = JWTAuth::parseToken()->refresh();
            return $this->createNewToken($newToken);
        } catch (\Exception $e) {
            return response()->json([
                'error' => 'Could not refresh token',
                'message' => $e->getMessage()
            ], 401);
        }
    }

    public function userProfile()
    {
        return response()->json(auth('api')->user());
    }

    protected function createNewToken($token)
    {
        return response()->json([
            'access_token' => $token,
            'token_type' => 'bearer',
            'expires_in' => JWTAuth::factory()->getTTL() * 60,
            'user' => Auth::user()
```

```
        ]);
    }

    // Admin only method

   public function adminDashboard()
    {
        // Gunakan pengecekan auth('api')->check() untuk menghindari error read
property of null
        if (!auth('api')->check() || auth('api')->user()->role !== 'admin') {
            return response()->json(['error' => 'Forbidden'], 403);
        }

        // Untuk memastikan apakah error bersumber dari hitung data (SQL vs Mongo):
        // Coba komentari baris stats terlebih dahulu untuk testing.
        return response()->json([
            'message' => 'Welcome to Admin Dashboard',
            'stats' => [
                'users' => User::count(),
                'admins' => User::where('role', 'admin')->count()
            ]
        ]);
    }

}
```

10. Buat Middleware Admin

    `php artisan make:middleware AdminMiddleware`

    Edit app/Http/Middleware/AdminMiddleware.php:

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class AdminMiddleware
{
    /**
     * Handle an incoming request.
     *
     * @param  \Closure(\Illuminate\Http\Request):
(\Symfony\Component\HttpFoundation\Response)  $next
     */

public function handle(Request $request, Closure $next): Response
    {
        if (!auth('api')->check() || auth('api')->user()->role !== 'admin') {
            return response()->json(['error' => 'Forbidden. Admin access
required.'], 403);
        }
        return $next($request);


    }}
```

11. Daftarkan middleware di app/Http/Kernel.php:

    Pada versi terbaru, ini tidak usah dibuat

```
protected $routeMiddleware = [
        // ...
        'admin' => \App\Http\Middleware\AdminMiddleware::class,
];
```

Pada versi baru
Boostrap\app.php

```
->withMiddleware(function (Middleware $middleware): void {
    $middleware->alias([
    'admin' => \App\Http\Middleware\AdminMiddleware::class,
    ]);
  })
```

12. Konfigurasi Auth Guard
    a. Buka file config/auth.php
    b. Ubah konfigurasi default guard dan providers:

```
'defaults' => [
    'guard' => 'api',
    'passwords' => 'users',
],

'guards' => [
    'web' => [
        'driver' => 'session',
        'provider' => 'users',
    ],

    'api' => [
        'driver' => 'jwt',
        'provider' => 'users',
    ],
],
```

13. Routes/api.php (jika tidak ada silakan buat dengan perintah: **`php artisan install:api`**

```
use App\Http\Controllers\AuthController;

Route::group([
    'prefix' => 'auth'
], function ($router) {
    Route::post('/login', [AuthController::class, 'login']);
    Route::post('/register', [AuthController::class, 'register']);
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::post('/refresh', [AuthController::class, 'refresh']);
    Route::get('/user-profile', [AuthController::class, 'userProfile']);
});

Route::middleware('auth:api')->group(function () {
    Route::get('/user-dashboard', function () {
        return response()->json(['message' => 'Welcome to User Dashboard']);
    });

    Route::group(['middleware' => 'admin'], function () {
```

```
        Route::get('/admin-dashboard', [AuthController::class, 'adminDashboard']);
    });
});
```

14. Buat Seeder untuk Testing

    php artisan make:seeder UserSeeder

    Edit database/seeders/UserSeeder.php:

```
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Support\Facades\Hash;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class UserSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        User::create([
            'name' => 'Admin User',
            'email' => 'admin@example.com',
            'password' => Hash::make('admin123'),
            'role' => 'admin'
        ]);

        User::create([
            'name' => 'Regular User',
            'email' => 'user@example.com',
            'password' => Hash::make('user123'),
            'role' => 'user'
        ]);
    }
}
```

    Jalankan seeder:

    php artisan db:seed --class=UserSeeder

15. Pastikan konfigurasi database di .env sudah benar

    Jalankan migrasi: **php artisan migrate**

16. Jalankan: **php artisan serve --port=8000**

17. End-Point

    1. **(USER) POST: http://localhost:8000/api/auth/register**

       Body → Raw (JSON)

```
{
    "name": "Test User",
    "email": "yere@example.com",
    "password": "yere123",
    "password_confirmation": "yere123"
}
```

2. **(USER) POST: http://localhost:8000/api/auth/login**
   Body → Raw (JSON)

```
{
   "email": "yere@example.com",
    "password": "yere123"
}
```

   Response:

```
{
    "access_token": "eyJ0eXAiOiJKV1QiLCJhbG … ",
    "token_type": "bearer",
    "expires_in": 3600,
    "user": {
        "id": 4,
        "name": "John Doe",
        "email": "yere@example.com",
        "email_verified_at": null,
        "created_at": "2025-06-13T09:51:23.000000Z",
        "updated_at": "2025-06-13T09:51:23.000000Z"
    }
}
```

   Gunakan/copy token ini untuk endpoint lain dengan header

3. **(USER) GET: http://localhost:8000/api/auth/user-profile**
   Authorization: Bearer token → isi dengan token

4. **(ADMIN) register sudah lewat seeder**

5. **(ADMIN) POST: http://localhost:8000/api/auth/login**
   Body → Raw (JSON)

```
{
    "email": "admin@example.com",
    "password": "admin123"
}
```

   Response:

```
{
    "access_token": "eyJ0eXAiOiJKV1QiLCJhbG … ",
    "token_type": "bearer",
    "expires_in": 3600,
    "user": {
        "id": 2,
        "name": "Admin User",
        "email": "admin@example.com",
        "email_verified_at": null,
        "created_at": "2025-06-14T02:04:32.000000Z",
        "updated_at": "2025-06-14T02:04:32.000000Z",
        "role": "admin"
    }
}
```

6. **(ADMIN) GET: http://localhost:8000/api/admin-dashboard**
   Authorization: Bearer token → isi dengan token

7. **POST: http://localhost:8000/api/auth/refresh**
   Authorization: Bearer token → isi dengan token

8. **POST: http://localhost:8000/api/auth/logout**
   Authorization: Bearer token → isi dengan token